

Лабораторная работа №14

Операционные системы

Панина Ж. В.

14 мая 2025

Российский университет дружбы народов, Москва, Россия

Информация

- Панина Жанна Валерьевна
- НКАбд-02-24, студ. билет № 1132246710
- студент направления “Компьютерные и информационные науки”
- Российский университет дружбы народов
- 1132246710@pfur.ru
- https://github.com/zvpanina/study_2024-2025_os-intro

Вводная часть

В условиях стремительного развития информационных технологий и широкого распространения операционных систем семейства UNIX (включая Linux и macOS), навыки программирования в командной оболочке становятся особенно востребованными. Скрипты оболочки позволяют автоматизировать рутинные задачи, управлять процессами и файлами, разрабатывать инструменты системного администрирования и DevOps. Умение создавать более сложные командные файлы с использованием логических конструкций и циклов существенно повышает эффективность работы специалистов и обеспечивает гибкость в решении типовых и нетиповых задач. Поэтому изучение этих основ является важным шагом для подготовки квалифицированных IT-кадров.

Объект исследования:

Процессы автоматизации типовых операций в операционной системе UNIX с помощью командной оболочки (shell).

Предмет исследования:

Средства и методы программирования в оболочке UNIX, включая использование условных операторов, циклов и других логических конструкций в скриптах.

Цель работы - изучить основы программирования в оболочке ОС UNIX/Linux. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задачи:

1. Написать командный файл, реализующий упрощённый механизм семафоров.

Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
3. Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

Материалы:

- командная оболочка `bash`;
- встроенные команды UNIX (`if`, `for`, `while`, `case`, `read`, `echo`, `test` и др.);
- учебные материалы и примеры скриптов;
- дистрибутив Fedora Linux, установленный в виртуальной среде VirtualBox.

Методы:

- практическое написание и тестирование shell-скриптов;
- пошаговая отладка командных файлов;
- анализ логики выполнения программных конструкций;
- сравнение поведения разных вариантов управляющих конструкций в скриптах.

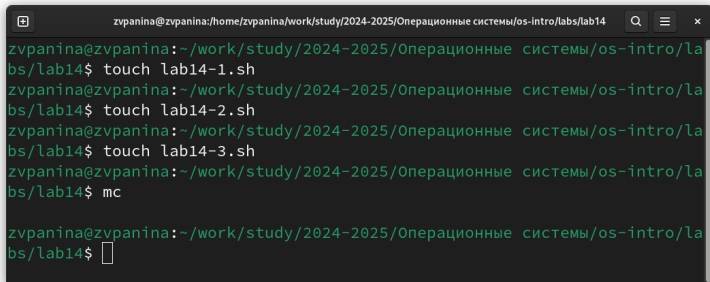
Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: - оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; - C-оболочка (или csh) — надстройка на оболочке Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд; - оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна;

- BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна.

Выполнение лабораторной работы

1. Создаю три файла .sh для скриптов и открываю mc для редактирования.



```
zvpanina@zvpanina:/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ touch lab14-1.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ touch lab14-2.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ touch lab14-3.sh
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ mc
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$
```

Рис. 1: Создание файлов

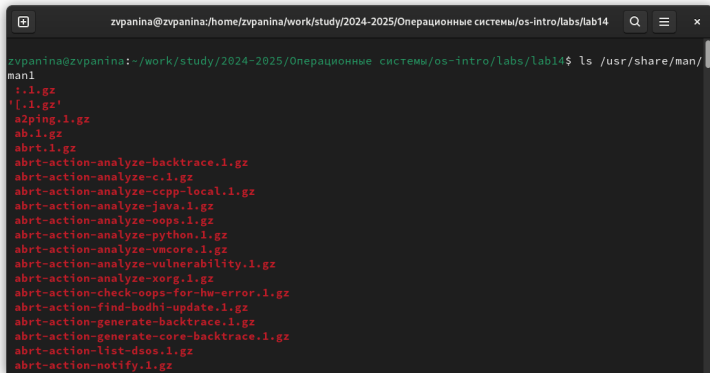
2. Пишу командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом).

```
mc [zvpanina@zvpanina]:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
lab14-1.sh [-M--] 4 L:[ 1+17 18/ 18] *(246 / 246b) <EOF>[↵][✕]
#!/bin/bash

lockfile="./lock.file"
exec {fn}>${lockfile}

while test -f "${lockfile}"
do
if flock -n ${fn}
then
echo "File is blocked"
sleep 5
echo "File is unlocked"
flock -u ${fn}
else
echo "File is blocked"
sleep 5
fi
done
```

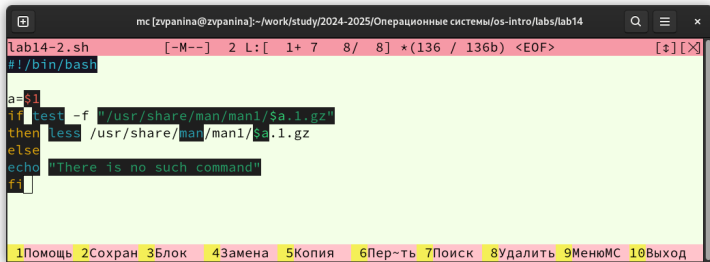
3. Изучаю содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд.



```
zvpanina@zvpanina:/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ ls /usr/share/man/man1
.:1.gz
'[:1.gz'
a2ping.1.gz
ab.1.gz
abrt.1.gz
abrt-action-analyze-backtrace.1.gz
abrt-action-analyze-c.1.gz
abrt-action-analyze-ccpp-local.1.gz
abrt-action-analyze-java.1.gz
abrt-action-analyze-oops.1.gz
abrt-action-analyze-python.1.gz
abrt-action-analyze-vmcore.1.gz
abrt-action-analyze-vulnerability.1.gz
abrt-action-analyze-xorg.1.gz
abrt-action-check-oops-for-hw-error.1.gz
abrt-action-find-bodhi-update.1.gz
abrt-action-generate-backtrace.1.gz
abrt-action-generate-core-backtrace.1.gz
abrt-action-list-dsos.1.gz
abrt-action-notify.1.gz
```

Рис. 3: Содержимое каталога `/usr/share/man/man1`

4. Реализую команду `man` с помощью командного файла. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.



```
mc [zvpanina@zvpanina]:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
lab14-2.sh [-M--] 2 L: [ 1+ 7 8/ 8] *(136 / 136b) <EOF> [?] [X]
#!/bin/bash

a=$1
if test -f "/usr/share/man/man1/$a.1.gz"
then less /usr/share/man/man1/$a.1.gz
else
echo "There is no such command"
fi

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пер~ть 7Поиск 8Удалить 9МенюМС 10Выход
```

Рис. 4: Второй скрипт

5. Используя встроенную переменную \$RANDOM, пишу командный файл, генерирующий случайную последовательность букв латинского алфавита.

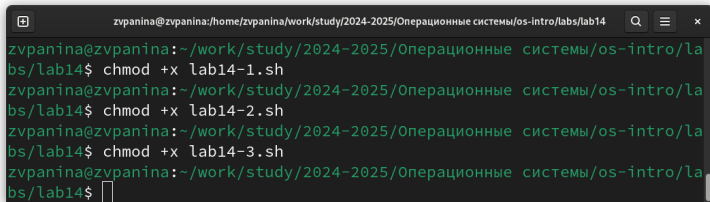


```
mc [zvpanina@zvpanina]:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
lab14-3.sh [-M--] 4 L: [ 1+16 17/ 17] *(538 / 538b) <EOF> [F][X]
#!/bin/bash
a=$1
for ((i=0; i<$a; i++))
do
    ((char=$RANDOM%26+1))
    case $char in
        1) echo -n a;; 2) echo -n b;; 3) echo -n c;; 4) echo -n d;; 5) echo -n e;;
        6) echo -n f;; 7) echo -n g;; 8) echo -n h;; 9) echo -n i;; 10) echo -n j;;
        11) echo -n k;; 12) echo -n l;; 13) echo -n m;; 14) echo -n n;; 15) echo -n o;;
        16) echo -n p;; 17) echo -n q;; 18) echo -n r;; 19) echo -n s;; 20) echo -n t;;
        21) echo -n u;; 22) echo -n v;; 23) echo -n w;; 24) echo -n x;; 25) echo -n y;;
        26) echo -n z;;
    esac
done
echo
```

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пер-ть 7Поиск 8Удалить 9МенюМС 10Выход

Рис. 5: Третий скрипт

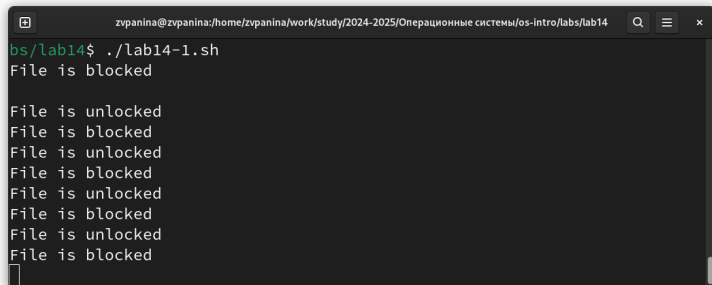
6. Добавляю права на исполнение для всех трёх файлов.

A terminal window with a dark background and light green text. The title bar at the top shows the user 'zvpanina' at host 'zvpanina' in the directory '/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14'. The terminal contains four lines of text: the first is the prompt 'zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14\$'; the next three are 'chmod +x lab14-1.sh', 'chmod +x lab14-2.sh', and 'chmod +x lab14-3.sh' respectively. The fourth line shows the prompt again with a cursor, indicating the command has been executed.

```
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$  
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ chmod +x lab14-1.sh  
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ chmod +x lab14-2.sh  
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ chmod +x lab14-3.sh  
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$
```

Рис. 6: Изменение прав доступа

7. Поочерёдно запускаю файлы. Так выглядит результат работы первого файла.



```
zvpanina@zvpanina:/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
bs/lab14$ ./lab14-1.sh
File is blocked

File is unlocked
File is blocked
File is unlocked
File is blocked
File is unlocked
File is blocked
File is unlocked
File is blocked
```

Рис. 7: Запуск первого файла

Запускаю второй файл с аргументом kill. Он выводит справку о данной команде.

```
zvpanina@zvpanina:/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14
ESC[4mKILLESC[24m(1) User Commands
ESC[4mKILLESC[24m(1)

ESC[1mNAMEESC[0m
kill - terminate a process

ESC[1mSYNOPSISESC[0m
ESC[1mkill ESC[22m[ESC[1m-ESC[4mESC[22msignalESC[24m|ESC[1m-s ESC[
[4mESC[22msignalESC[24m|ESC[1m-pESC[22m] [ESC[1m-q ESC[4mESC[22mvalueESC[
[24m] [ESC[1m-aESC[22m] [ESC[1m--timeoutESC[0m
ESC[4mmillisecondsESC[24m ESC[4msignalESC[24m] [ESC[1m--ESC[22m]
ESC[4mpidESC[24m|ESC[4mnameESC[24m...

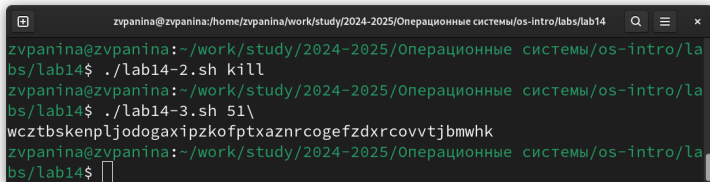
ESC[1mkill -l ESC[22m[ESC[4mnumberESC[24m] | ESC[1m-LESC[0m

ESC[1mDESCRIPTIONESC[0m
The command ESC[1mkill ESC[22msends the specified ESC[4msignalESC[
24m to the specified
processes or process groups.

If no signal is specified, the ESC[1mTERM ESC[22msignal is sent. T
he default
action for this signal is to terminate the process. This signal
should be used in preference to the ESC[1mKILL ESC[22msignal (numb
er 9),
:

```

Запускаю третий файл с указанием длины строки. Он выводит случайную последовательность букв латинского алфавита.

A terminal window with a dark background and green text. The title bar shows the user 'zvpanina' and the path '/home/zvpanina/work/study/2024-2025/Операционные системы/os-intro/labs/lab14'. The terminal shows the user running two scripts: './lab14-2.sh kill' and './lab14-3.sh 51\'. The second script outputs a long random string of lowercase letters: 'wcztbksenpljodogaxipzkoftpaznrcogefzdxrcovvtjbmwhk'.

```
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ ./lab14-2.sh kill
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$ ./lab14-3.sh 51\
wcztbksenpljodogaxipzkoftpaznrcogefzdxrcovvtjbmwhk
zvpanina@zvpanina:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab14$
```

Рис. 9: Запуск третьего файла

Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке: `1 while [$1 != "exit"]`
 - В данной строчке допущены следующие ошибки: не хватает пробелов после первой скобки [и перед второй скобкой] выражение \$1 необходимо взять в "", потому что эта переменная может содержать пробелы Таким образом, правильный вариант должен выглядеть так: `while [ "$1" != "exit" ]`
2. Как объединить (конкатенация) несколько строк в одну?
 - Чтобы объединить несколько строк в одну, можно воспользоваться несколькими способами: Первый: `VAR1="Hello," VAR2=" World" VAR3="$VAR1$VAR2" echo "$VAR3" : Hello, World : VAR1 = "Hello," VAR1+= "World" echo "$VAR1"`
Результат: Hello, World

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash?

- Команда seq в Linux используется для генерации чисел от ПЕРВОГО до ПОСЛЕДНЕГО шага INCREMENT. Параметры: seq LAST: если задан только один аргумент, он создает числа от 1 до LAST с шагом шага, равным 1. Если LAST меньше 1, значение is не выдает. seq FIRST LAST: когда заданы два аргумента, он генерирует числа от FIRST до LAST с шагом 1, равным 1. Если LAST меньше FIRST, он не выдает никаких выходных данных. seq FIRST INCREMENT LAST: когда заданы три аргумента, он генерирует числа от FIRST до LAST на шаге INCREMENT. Если LAST меньше, чем FIRST, он не производит вывод. seq -f «FORMAT» FIRST INCREMENT LAST: эта команда используется для генерации последовательности в форматированном виде. FIRST и INCREMENT являются необязательными. seq -s «STRING» ПЕРВЫЙ ВКЛЮЧЕНО: Эта команда используется для STRING для разделения чисел. По умолчанию это значение равно /n. FIRST и INCREMENT являются необязательными. seq -w FIRST INCREMENT LAST: эта команда используется для выравнивания ширины путем заполнения начальными нулями. FIRST и INCREMENT являются необязательными.

4. Какой результат даст вычисление выражения $\$(10/3)$?

- Результатом данного выражения $\$(10/3)$ будет 3, потому что это целочисленное деление без остатка.

5. Укажите кратко основные отличия командной оболочки zsh от bash.

- Отличия командной оболочки zsh от bash: В zsh более быстрое автодополнение для cd с помощью Tab В zsh существует калькулятор zcalc, способный выполнять вычисления внутри терминала В zsh поддерживаются числа с плавающей запятой В zsh поддерживаются структуры данных «хэш» В zsh поддерживается раскрытие полного пути на основенеполных данных В zsh поддерживается замена части пути В zsh есть возможность отображать разделенный экран, такой же как разделенный экран vim

6. Проверьте, верен ли синтаксис данной конструкции: `for ((a=1; a <= LIMIT; a++))`
- Синтаксис данной конструкции верен, потому что, используя двойные круглые скобки, можно не писать `$` перед переменными `()`.
7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки?
- Преимущества и недостатки скриптового языка `bash`:
 - Один из самых распространенных и ставится по умолчанию в большинстве дистрибутивах Linux, MacOS
 - Удобное перенаправление ввода/вывода

- Большое количество команд для работы с файловыми системами Linux
- Можно писать собственные скрипты, упрощающие работу в Linux Недостатки скриптового языка bash:
- Дополнительные библиотеки других языков позволяют выполнить больше действий
- Bash не является языком общего назначения
- Утилиты, при выполнении скрипта, запускают свои процессы, которые, в свою очередь, отражаются на скорости выполнения этого скрипта
- Скрипты, написанные на bash, нельзя запустить на других операционных системах без дополнительных действий

Результаты

В ходе выполнения лабораторной работы я изучила основы программирования в оболочке ОС UNIX/Linux, научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.